POLITECNICO
MILANO 1863

Prova Finale
Progetto di Reti Logiche

Christian Salvini

Codice persiona: 10796087

Matricola: 982602

Corso di Reti Logiche

2024/25

Professore: Palermo Gianluca

Contents

1 Introduzione **2**
1.1 Esempio 2

2 Architettura **4**
2.1 Datapath 4
2.1.1 datapath_counter 4
2.1.2 datapath_address 4
2.1.3 datapath_core 5
2.2 FSM 7

3 Risultati sperimentali **10**
3.1 Sintesi 10
3.2 Test 10

4 Conclusioni **12**

Progetto RL

salvoc878

August 2025

1 Introduzione

Questo documento costituisce la relazione della "Prova Finale (Progetto di Reti Logiche)" per l'anno accademico 2024/2025. Si chiede di realizzare in linguaggio VHDL un modulo HW in grado di leggere, elaborare e salvare una sequenza di dati all'interno di una memoria. L'elaborazione dei dati consiste nell'applicazione di un filtro differenziale a un insieme di valori numerici.

All'inizio della sequenza di dati ci sono 17 byte di configurazione:

- 2 byte che rappresentano la lunghezza della sequenza da elaborare
- 1 byte che rappresenta quale filtro utilizzare (filtro di ordine 3 o ordine 5)
- 14 byte con i coefficienti dei due tipi di filtro. Se il filtro è di ordine 3 vengono usati i primi 7 byte, altrimenti i restanti 7

Subito dopo il 17 byte inizia la sequenza di parole W con lunghezza K.

Il filtro differenziale viene applicato per ogni elemento della sequenza: ogni valore viene combinato con quelli che lo precedono e lo seguono con i coefficienti letti da memoria secondo la seguente formula:

$$R(i) = \frac{1}{n} \sum_{k=-l}^{l} C(k) * W(k+i) \tag{1}$$

dove:

- per il filtro di ordine 3: l=2, n = 12
- per il filtro di ordine 5: l=3, n = 60
- R(i) è il valore filtrato dell'i-esima parola W

Il valore R viene scritto nell'indirizzo di memoria successivo all'ultima parola W.

1.1 Esempio

Di seguito viene fornito un esempio del calcolo della prima parola W della sequenza nel caso in cui il filtro sia di ordine 5.

Vengono letti da memoria i seguenti 17 byte di configurazione:

0x00	0x18	0x01	0x00	0xff	0x08	0x00	0xf8	0x01	0x00	0x01	0xf7	0x2d	0x00	0xd3	0x09	0xff
0	24	1	0	-1	8	0	-8	1	0	1	-9	45	0	-45	9	-1

quindi:

- $k = 0x00$ & $0x18 = 0x0018 \rightarrow 24$
- $s = 1 \rightarrow$ filtro di ordine 5
- c = [0x01, 0xf7, 0x2d, 0x00, 0xd3, 0x09, 0xff] = [1, -9, 45, -0,-45, 9, -1]

La sequenza di parole è la seguente:

0x20	0xe8	0xdd	0x00	0xf2e	0xca	0xd9	...
32	-24	-35	0	46	-43	-39	...

Vengono lette le prime 4 parole W così che in w_values(3) sia presente 0x20
w_values = [0x00, 0xdd, 0xe8, 0x20, 0x00, 0x00, 0x00] = [0, -35, -24, 32, 0, 0, 0]

$$\begin{aligned} sum &= 00 * 01 + 00 * f7 + 00 * 2d + 20 * 00 + e8 * d3 + dd * 09 + 00 * ff \\ &= 0x0438 + 0xfec5 \\ &= 0x002fd\ (18bit) \end{aligned} \tag{2}$$

il fattore di normalizzazione 1/60 viene approssimato con 1/64 + 1/1024

$$\begin{aligned} 1/64 \rightarrow sum_shift6 &= sum >> 6 + 0 \\ 00\ 0000\ 0010\ 1111\ 1101 &>> 6 + 0 \\ &= 00\ 0000\ 0000\ 0000\ 1011 \\ &= 0x0000b \end{aligned} \tag{3}$$

$$\begin{aligned} 1/1024 \rightarrow sum_shift10 &= sum >> 10 + 0 \\ 00\ 0000\ 0010\ 1111\ 1101 &>> 10 + 0 \\ 00\ 0000\ 0000\ 0000\ 0000 \\ &= 0x00000 \end{aligned} \tag{4}$$

R = $0x0000b$ + $0x00000 = 0x0000b$
il valore R viene troncato a 8 bit: o_mem_data = $0x0b \rightarrow 12_{dec}$

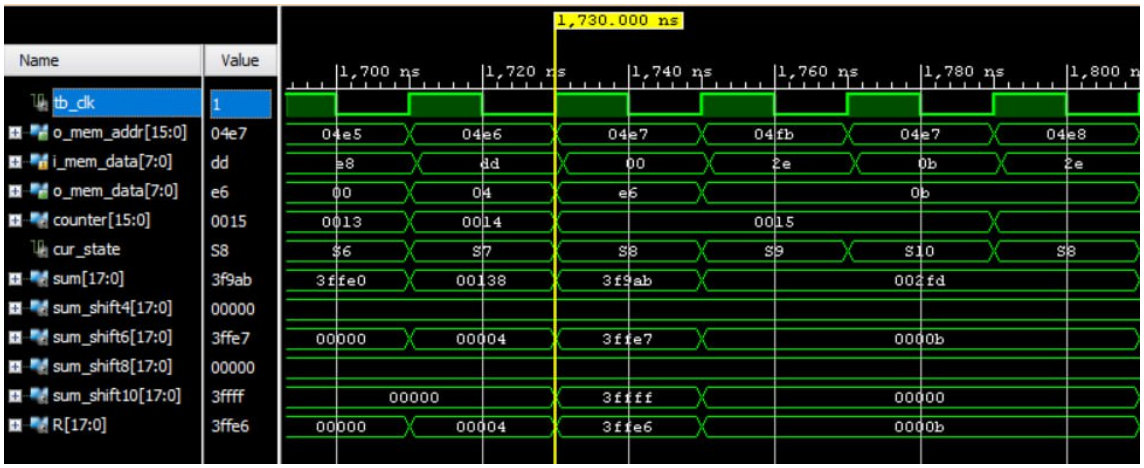

Figure 1: Risultato esempio

2 Architettura

La struttura principale del progetto è organizzata in DATAPATH + FSM, così da avere una separazione netta tra la logica di elaborazione dei dati (datapath) dalla logica di controllo.

2.1 Datapath

Il datapath è costituito da 3 moduli:

2.1.1 datapath_counter

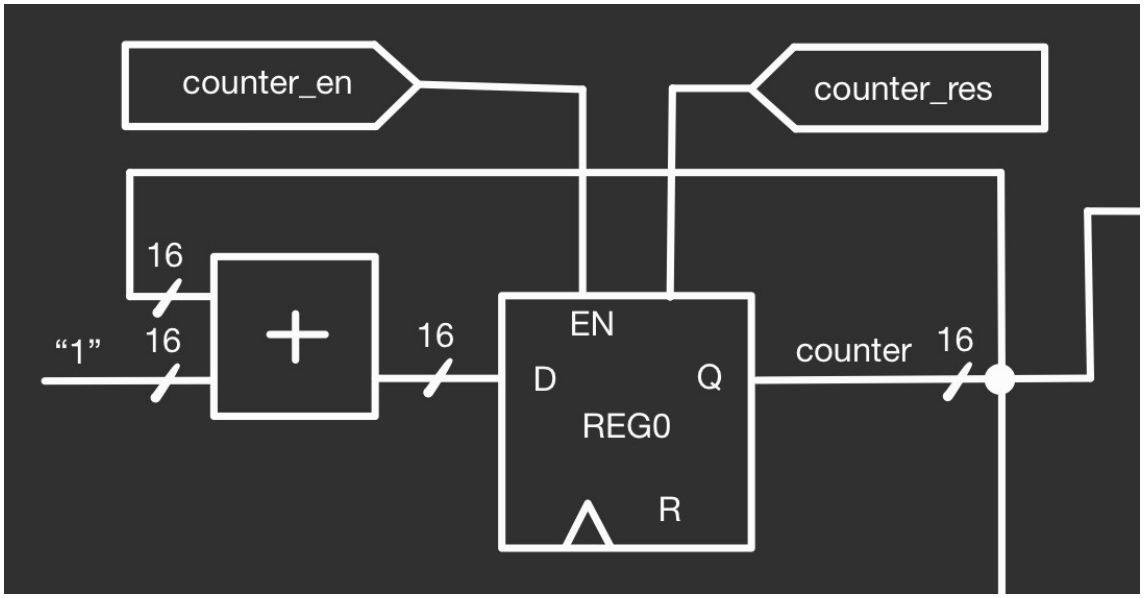

Figure 2: datapath_counter

Implementa un semplice contatore a 16 bit ed è costituito da un unico registro sincrono che, quando abilitato (`counter_en = '1'`), viene incrementato di 1 sul fronte di salita di `i_clk`. Ha due segnali di reset asincroni (`i_rst` e `counter_res`) che, quando posti a '1', azzerano il contatore. Il valore interno `o_reg0` che contiene il valore del contatore, è esposto sull'uscita `counter`.

2.1.2 datapath_address

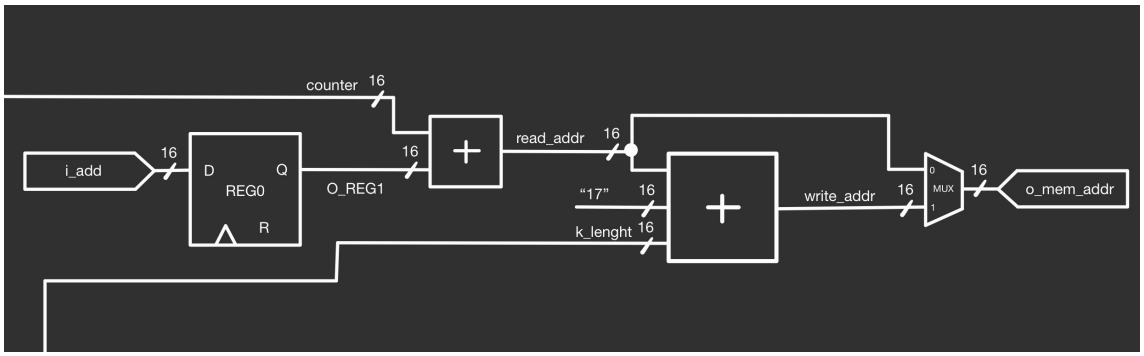

Figure 3: datapath_address

Questo modulo serve per calcolare sia l'indirizzo di lettura che l'indirizzo di scrittura della memoria. Composto da un registro sincrono che salva l'indirizzo posto in ingresso sul segnale i_add. Con della logica combinatoria vengono calcolati:

- `read_addr` → l'indirizzo della memoria da cui leggere le parole W.

- `write_addr` → l'indirizzo della memoria in cui scrivere i risultati R.

In uscita è presente un multiplexer con segnale di controllo `mux_addr`:

- se `mux_addr = '0'` sull'uscita `o_mem_addr` viene posto il valore di `read_addr`
- se `mux_addr = '1'` sull'uscita `o_mem_addr` viene posto il valore di `write_addr`

2.1.3 datapath_core

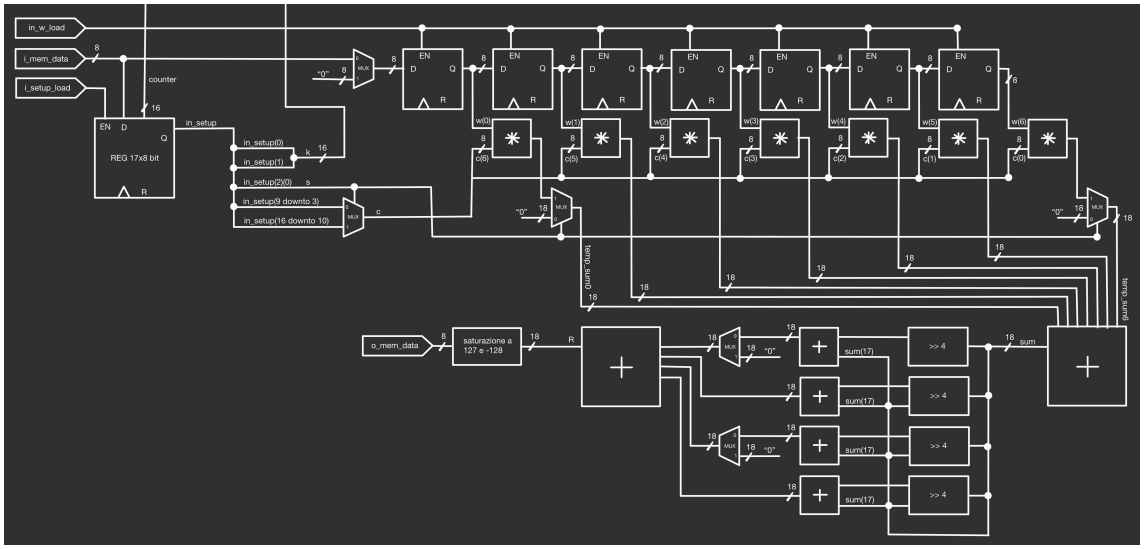

Figure 4: datapath_core

Il datapath principale che si occupa di leggere da memoria i byte di configurazione, la sequenza di parole W e salvare in memoria i risultati R. Porte e segnali principali:

- **I/O dati:**
  - `i_mem_data (8 bit)` → dato in ingresso dalla memoria
  - `o_mem_data (8 bit)` → dato elaborato in uscita verso la memoria
- **clock e reset:**
  - `i_clk` → segnale di clock
  - `i_rst` → segnale di reset
- **segnali in ingresso:**
  - `counter (16 bit)` → contatore fornito da `datapath_counter`
  - `in_setup_load` → segnale di controllo che abilita la lettura delle parole di configurazione in `in_setup`
  - `in_w_load` → segnale di controllo che abilita la lettura delle parole W in `w_values`
- **segnali di uscita:**
  - `in_setup_end` → viene portato a '1' quando il caricamento di `in_setup` è completato
  - `in_w_end` → viene portato a '1' quando l'ultimo valore della sequenza di parole W è stato letto
  - `k_lenght` → valore di k, lunghezza della sequenza di parole W
- **segnali di uscita:**
  - `in_setup` → array di 17 registri da 8 bit contenente i byte di configurazione

- `w_values` → shift register composto da 7 registri da 8 bit contenente una finestra di 7 parole della sequenza di parole W
- `c` → array di segnali da 8 bit contente i coefficienti del filtro scelti in base a s
- `temp_sum0, temp_sum6, sum, sum_shift4, sum_shift6, sum_shift8, sum_shift10, R` → segnali intermedi per il calcolo

Comportamento del componente

- **Caricamento `in_setup`:**
  quando `in_setup` è abilitato (`in_setup_load = '1'`), viene memorizzato il byte letto da `i_mem_data` sul fronte di salita del clock e salvato nel registro corrispondente in base al valore del contatore `counter`.
  Quando `i_rst = '1'` vengono azzerati tutti i valori dei registri.

- **Calcolo valori di k e s:**
  - $k = in_setup(0)$ & $in_setup(1)$ → si ottiene la lunghezza della sequenza concatenando i byte K1 e K2
  - $s =$ LSB di `in_setup(2)` → selettore del filtro

- **Selezione coefficienti c:**
  - se $s =' 0'$ → vengono posti in c i valori da `in_setup(3)` a `in_setup(9)`, cioè i coefficienti del filtro di ordine 3 (da C1 a C7)
  - se $s =' 1'$ → vengono posti in c i valori da `in_setup(10)` a `in_setup(16)`, cioè i coefficienti del filtro di ordine 5 (da C8 a C14)

- **Caricamento `w_values`:**
  quando `w_values` è abilitato (`in_w_load = '1'`), viene memorizzato il byte letto da `i_mem_data` nel primo registro `w_values(0)`, mentre gli altri `w_values(i)` si aggiornano dal precedente.
  Quando `last_w = '1'` viene letto 0 invece che il valore di `i_mem_data` per permettere l'elaborazione delle ultime parole W (l'elaborazione dell'ultima parola W avviene quando raggiunge il registro `w_values(3)`).

- **Moltiplicazioni:**
  - `temp_sum0 = w_values(0)*c(6)`
  - `temp_sum6 = w_values(0)*c(6)`

  se il filtro è di ordine 3 (`s = '0'`) questi due segnali vengono posti a 0 per rispettare il numero di parole da sommare nel caso di filtro di ordine 3. Vengono moltiplicati i restanti valori `w_values` con i loro coefficienti `c` corrispondenti.

- **Somma:**
  tutti i valori moltiplicati nello stadio precedente vengono estesi a 18 bit e sommati nel segnale `sum`.

- **Shift:**
  sum viene shiftato a destra di 4, 6, 8, e 10 bit e viene sommato a ognuno il MSB di `sum`. Quindi, se sum è negativo, viene sommato +1 al risultato dei vari shift per compensare l'errore quando un numero negativo subisce shift multipli (`sum_shift4, sum_shift6, sum_shift8, sum_shift10`).
  Se il filtro è di ordine 5 (`s = '1'`) i valori di `sum_shift4` e `sum_shift8` vengono posti a 0.

- **Calcolo valore R:**
  il valore di R si ottiene sommando i valori `sum_shift4`, `sum_shift6`, `sum_shift8` e `sum_shift10`.

- **Saturazione:**
  - se `R > 127` → `o_mem_data = 127`
  - se `R < -128` → `o_mem_data = 128`
  - altrimenti viene posto il valore di R sull'uscita `o_mem_data` ridotto a 8 bit

2.2 FSM

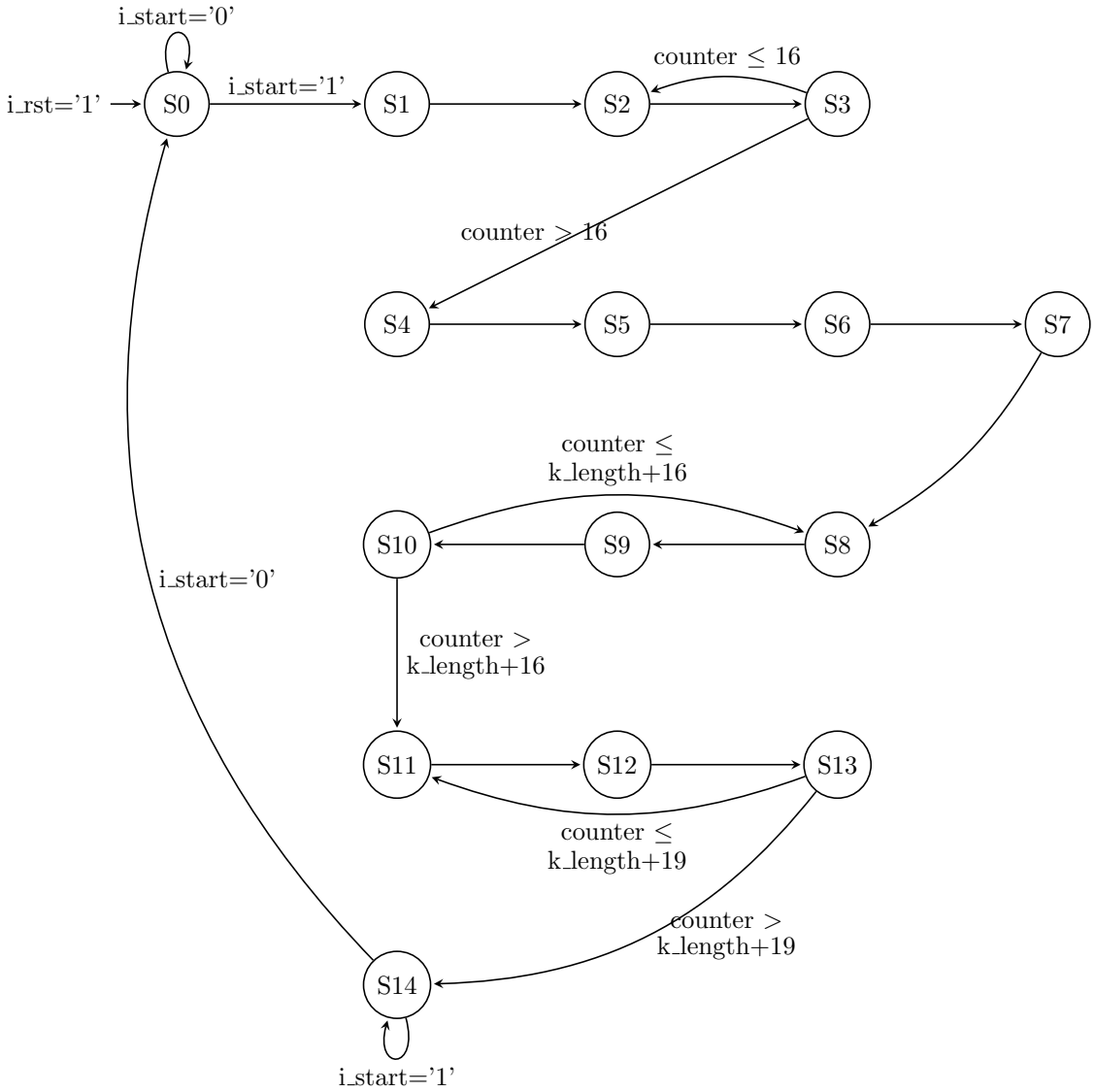

Figure 5: Diagramma della FSM. Tutti gli archi di i_rst sono stati omesssi

La FSM genera i segnali che controllano i vari datapath. Un processo cambia lo stato corrente della FSM allo stato successivo sul fronte di salita del clock e quando `i_rst = '1'` la riporta allo stato S0. Lo stato successivo e il valore dei vari segnali di controllo sono determinati da due processi distiniti. Gli stati che compongono la FSM sono 14 e sono i seguenti (i segnali di controllo che non compaioni sono da considerarsi uguali a '0'):

- **S0**
  Stato di riposo che viene raggiunto ogni volta che `i_rst = '1'`. Se `i_start = '1'` l'elaborazione inizia e si passa allo stato S1.

- **S1**

  - `o_mem_en = '1'` → viene attivata la memoria

  si passa allo stato S2.

- **S2 → S3 (loop)**
  loop in cui vengono lette le 17 parole di configurazione e salvate nei rispettivi registri del blocco `in_setup`.
  S2

- `o_mem_en = '1'`
- `counter_en = '1'` → viene attivato il counter, quindi sul prossimo fronte di salita del clock counter viene aumentato di 1
- `in_setup_load = '1'` → viene attivato `in_setup`, quindi sul prossimo fronte di salita del clock viene letta la parola presente al momento sul segnale `i_mem_data`

S3

- `o_mem_en = '1'`

se `counter > 16` è stata letta l'utlima parola di configurazione, quindi si passa allo stato S4. Altrimenti il loop continua.

- **S4**
  è uno stato di transizione tra la lettura delle parole di configurazione e l'inizio della lettura delle parole W.

  - `counter_en = '1'` → viene attivato il counter, quindi sul prossimo fronte di salita del clock counter viene aumentato di 1

  si passa allo stato S5.

- **S5 → S6 → S7 (sequenza)**
  sequenza di 3 stati in cui `w_values` legge le prime 3 parole W della sequenza senza salvataggio del valore R perchè la prima parola W deve trovarsi nel registro `w_values(3)`.

  - `o_mem_en = '1'`
  - `counter_en = '1'` → viene attivato il counter, quindi sul prossimo fronte di salita del clock counter viene aumentato di 1
  - `in_w_load = '1'` → viene attivato `w_values`, quindi sul prossimo fronte di salita del clock viene letta la parola W presente al momento sul segnale `i_mem_data`

  alla fine della sequenza si passa allo stato S8.

- **S8 → S9 → S10 (loop)**
  loop in cui viene letta una parola W (se è la prima iterazione viene letta la quarta parola W, quindi nella posizione `w_values(3)` è presente la prima parola W), calcolato il valore R e salvato nella memoria.
  S8

  - `o_mem_en = '1'`
  - `in_w_load = '1'` → viene attivato `w_values`, quindi sul prossimo fronte di salita del clock viene letta la parola W presente al momento sul segnale `i_mem_data`

  S9

  - `o_mem_en = '1'`
  - `o_mem_we = '1'` → la memoria viene posta in modalità scrittura, quindi sul prossimo fronte di salita del clock viene scritto il valore posto su `o_mem_data` all'indirizzo `o_mem_addr`
  - `mux_addr = '1'` → sul segnale `o_mem_addr` viene posto l'indirizzo `write_addr`

  S10

  - `o_mem_en = '1'`
  - `counter_en = '1'` → viene attivato il counter, quindi sul prossimo fronte di salita del clock counter viene aumentato di 1

  se `counter > k_lenght + 16` è stata letta l'ultima parola W della sequenza, quindi si passa allo stato S11. Altrimenti il loop continua.

- **S11 → S12 → S13 (loop)**
  loop in cui vengono lette le ultime 3 parole W così che all'ultima iterazione l'ultima parola W si trova nella posizione `w_values(3)`
  S11
  - `o_mem_en = '1'`
  - `in_w_load = '1'` → viene attivato `w_values`, quindi sul prossimo fronte di salita del clock viene letta la parola W presente al momento sul segnale `i_mem_data`
  - `last_w = '1'` → `w_values` legge il valore 0 invece che quello presente sul segnale `i_mem_data`

  S12
  - `o_mem_en = '1'`
  - `o_mem_we = '1'` → la memoria viene posta in modalità scrittura, quindi sul prossimo fronte di salita del clock viene scritto il valore posto su `o_mem_data` all'indirizzo `o_mem_addr`
  - `mux_addr = '1'` → sul segnale `o_mem_addr` viene posto l'indirizzo `write_addr`
  - `last_w = '1'`

  S13
  - `o_mem_en = '1'`
  - `counter_en = '1'` → viene attivato il counter, quindi sul prossimo fronte di salita del clock counter viene aumentato di 1
  - `last_w = '1'`

  se `counter > k_lenght + 19` è stato salvato in memoria il valore R dell'ultima parola W, si passa allo stato S14. Altrimenti il loop continua.

- **S14**
  - `o_done = '1'`
  - `counter_res = '1'` → viene resettato il counter per future elaborazioni.

  Il calcolo dei valori R è terminato. Se `i_start = '0'` l'elaborazione termina e si torna allo stato S0 in attesa di una nuova elaborazione (lo stato S14 non passa allo stato S0 fino a che `i_start` non viene posto a 0.

3 Risultati sperimentali

3.1 Sintesi

Il processo di sintesi è stato completato con successo e senza errori, dimostrando che il codice VHDL sviluppato rispetta i requisiti di sintetizzabilità e può essere correttamente tradotto in hardware fisico. In particolare, come evidenziato dalla tabella, non sono presenti latch indesiderati

Table 1: Utilizzo delle Risorse Logiche - Slice Logic

Site Type	**Used**	**Fixed**	**Available**	**Util%**
Slice LUTs*	815	0	41000	1.99
LUT as Logic	815	0	41000	1.99
LUT as Memory	0	0	13400	0.00
Slice Registers	224	0	82000	0.27
Register as Flip Flop	224	0	82000	0.27
Register as Latch	0	0	82000	0.00
F7 Muxes	0	0	20500	0.00
F8 Muxes	0	0	10250	0.00

che rappresenterebbero problemi durante la simulazione post-sintesi. Il numero di registri che si ottengono a priori analizzando il codice VHDL sono:

$$\begin{aligned} 4 + (15\ stati\ della\ FSM) \\ 16 + (counter) \\ 16 + (address) \\ 136 + (in_setup) \\ 56 + (w_values) \\ = 228 flipflop \end{aligned} \tag{5}$$

Nella tabella, invece, viene riportato l'utilizzo di 224 flip-flop. Quindi, il sintetizzatore ha ottimizzato la sintesi dei registri.

3.2 Test

Tutti i testi elencati in questo paragrafo sono stati provati e passati con successo sia in "Behavioral Simulation" che in "Post-Synthesis Functional Simulation". Oltre al test bench fornito dal docente, sono stati individuati i seguenti casi particolari:

- **Massimo valore possibile**
  sono stati forniti i seguenti valori del filtro di ordine 5 e della sequenza di parole W per testare la gestione del valore massimo ottenibile durante l'elaborazione.

  - c = [0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80] = [-128, -128, -128, -128,-128, -128, -128]
  - w = [0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80] = [-128, -128, -128, -128,-128, -128, -128]
  - valore ottenuto = 127

  il modulo non va in overflow e il valore ottenuto viene correttamente saturato a 127.

- **Minimo valore possibile**
  sono stati forniti i seguenti valori del filtro di ordine 5 e della sequenza di parole W per testare la gestione del valore minimo ottenibile durante l'elaborazione.

  - c = [0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80] = [-128, -128, -128, -128,-128, -128, -128]
  - w = [0x7f, 0x7f, 0x7f, 0x7f, 0x7f, 0x7f, 0x7f] = [127, 127, 127, 127,127, 127, 127]
  - valore ottenuto = -128

  il modulo non va in overflow e il valore ottenuto viene correttamente saturato a -128.

- **Lunghezza sequenza di parole W minima**
  è stato testato il comportamento del modulo nel caso in cui la lunghezza della sequenza di parole W sia uguale a 7..

- **Segnale di reset asincrono** durante l'elaborazione viene alzato il reset e il modulo si comporta correttamente:
  - la FSM ritorna allo stato S0
  - i segnali di controllo della FSM vengono riportati a 0
  - il contatore viene azzerato
  - i registri `in_setup e w_values` vengono azzerati

- **Elaborazioni multiple** è stato testato il comportamento del modulo nel caso in cui alla fine di un'elaborazione ne inizi un'altra senza prima alzare il segnale di reset. Non sono stati riscontrati problemi e in particolare il counter viene azzerato correttamente e i dati dell'elaborazione precedente vengono sovrascritti completamente.

4 Conclusioni

Lo sviluppo del modulo VHDL ha raggiunto gli obiettivi progettuali prefissati, completando con esito positivo le tre fasi di verifica: la simulazione comportamentale, la fase di sintesi conclusa senza generare errori o avvisi critici e la simulazione post-sintesi.

Osservazioni sull'architettura del modulo:

1. il calcolo di `sum` viene eseguito a 18 bit per evitare di incorrere in overflow durante la somma. Infatti:

   - il valore massimo ottenibile é $(-128) * (-128) * 7 = 16384 * 7 = 114688$
   - il valore minimo ottenibile é $(-128) * (127) * 7 = -16256 * 7 = -113792$

   entrambi i numeri sono rappresentabili con 18 bit in complemento a 2.

2. secondo la specifica tra un'elaborazione e l'altra non viene dato il segnale di reset, ciò comporta il fatto che non vengono resettati i registri e il counter. Per questo motivo nel datapath_counter è stato inserito un ulteriore segnale di reset (`counter_res`) per riportare il valore di `counter` a 0 una volta termina l'elaborazione. La stessa cosa non viene fatta per i registri perchè venendo completamente sostituiti dai nuovi valori letti un possibile problema di sovrapposizione non si presenta. In particolare, nello shift_register, alla fine di un'elaborazione sono sempre presenti degli 0 nelle posizioni `w_values(0), w_values(1) e w_values(2)` che si spostano nelle posizioni `w_values(4), w_values(5) e w_values(6)` nel momento in cui nella posizione `w_values(3)` si trova la prima parola W della nuova elaborazione. Dato che, la lunghezza minima della sequenza è 7, i valori vecchi vengono sempre sovrascritti.

3. il valore R che in un dato istante viene posto in uscita sul segnale `o_mem_data` si riferisce al valore W che in quel momento si trova nel registro. `w_values(3)`